
بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



Natural Language Processing

Lecture – Artificial Neural Networks

Author: Dr. Rao Muhammad Adeel Nawab

Instructor: Mr. Muhammad Ehsan

Lecture Outline

- ① Recap**
- ② Artificial Neural Networks**
- ③ General Architecture - Artificial Neural Networks**
- ④ Perceptron - Two Layer Artificial Neural Networks**
- ⑤ Machine Learning Cycle – Perceptron**
- ⑥ Multi-layer Artificial Neural Networks**
- ⑦ Overfitting – Artificial Neural Networks**

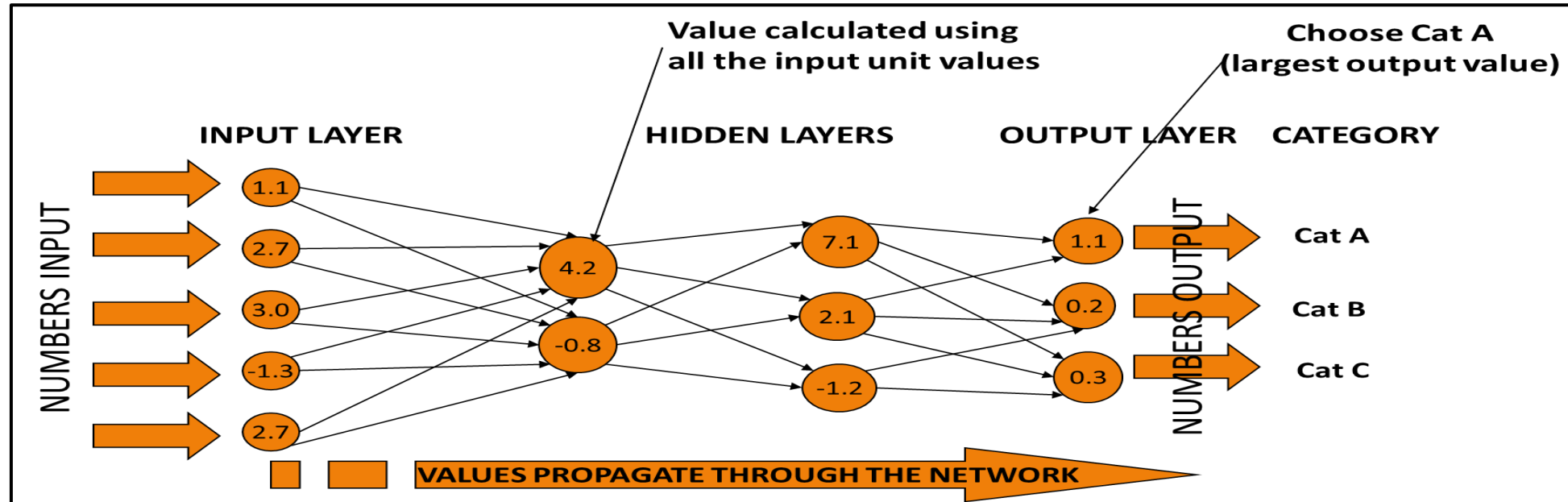
What are Neural Networks ?

- An **inspiration from biological neural systems**, the most robust learning systems we know.
- Attempt to understand natural biological systems through computational modeling.
- Neural networks in artificial intelligence draw significant inspiration from biological neural systems, particularly the human brain. The basic building blocks of both systems are neurons: biological neurons in the human brain and artificial neurons in computational models.

Neural Network Learning

- Learning approach of NN algorithm based on modeling while learning in biological neural systems is based on **adaptation**.
- Two main algorithms:
 - **Perceptron**: Initial algorithm for learning simple neural networks (with no hidden layer) developed in the 1950's.
 - **Backpropagation**: More complex algorithm for learning multi-layer neural networks developed in the 1980's.

General Architecture - Artificial Neural Networks



Three Main Layers



- » Input Layer
- » Hidden Layer
- » Output Layer

General Architecture - Artificial Neural Networks (Cont.)



Each Layer contains one or more Units



Input Units

» Units in the Input Layer called Input Units



Hidden Units

» Units in the Hidden Layer called Hidden Units



Output Units

» Units in the Output Layer called Output Units

Number of Units at each Layer (Cont.)

Question 03

» How many **Units** should be there at Hidden Layer?

Answer

» There is **no definite answer** to this question

» Normally, people **randomly** select

- ❑ Number of Hidden Layers and
- ❑ Number of Hidden Units

SoftMax Layer - ANN



Considering Vehicle Categorization Problem

» **Output Units** contain the **Output generated by ANN**



Problem

» How can we **interpret** the **vectors at Output Units** to **categorize Image** as Car / Bus / Tank?



A Possible Solution

» Use a Softmax Function

SoftMax Layer – ANN (Cont.)



Softmax Function

- » Softmax Function takes **vectors** as input and **converts** them into **probabilities** such that **sum of probabilities of all Output Units is 1**
- » The Output Unit with the **highest probability** will be the **predicted Class / Category** of an instance / example



Softmax Layer is the **last** Layer of ANN

Mathematical Functions – ANNs



ANNs **embed** giant Mathematical Functions (e.g. relu, sigmoid etc.)



All the Hidden Units and Output Units in an ANN, have the
» **Same** Mathematical Function



Input to a Mathematical Function
» Weighted Sum of Inputs (S)

Mathematical Functions – Regular Neural Network



Some of the **popular** and **widely used** Mathematic Functions in Regular Neural Network are

» Sigmoid



Formula of Sigmoid Function

»
$$\sigma(S) = \frac{1}{(1 + e^{-S})}$$

» where 'S' is Weighted sum of Inputs

Fully Connected ANN vs Partially Connected ANN



Fully Connected ANN

» A Fully Connected Neural Network consists of a **series** of Fully Connected Layers



Fully Connected Layer

» In a Fully Connected Layer, **each Unit** receives input from **every Unit** of the previous layer



Partially Connected ANN

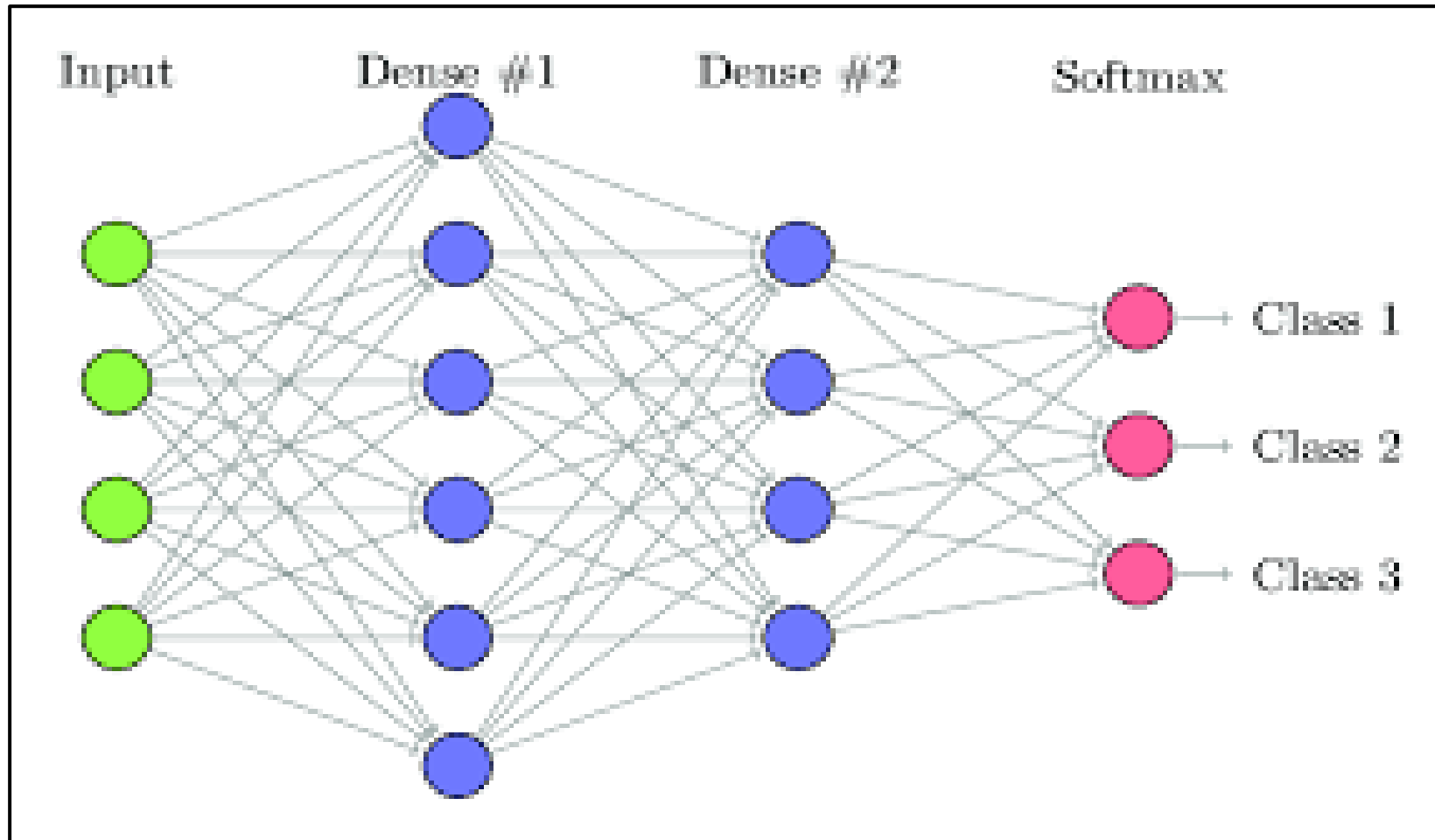
» A Partially Connected Neural Network consists of a **series** of Partially Connected Layers



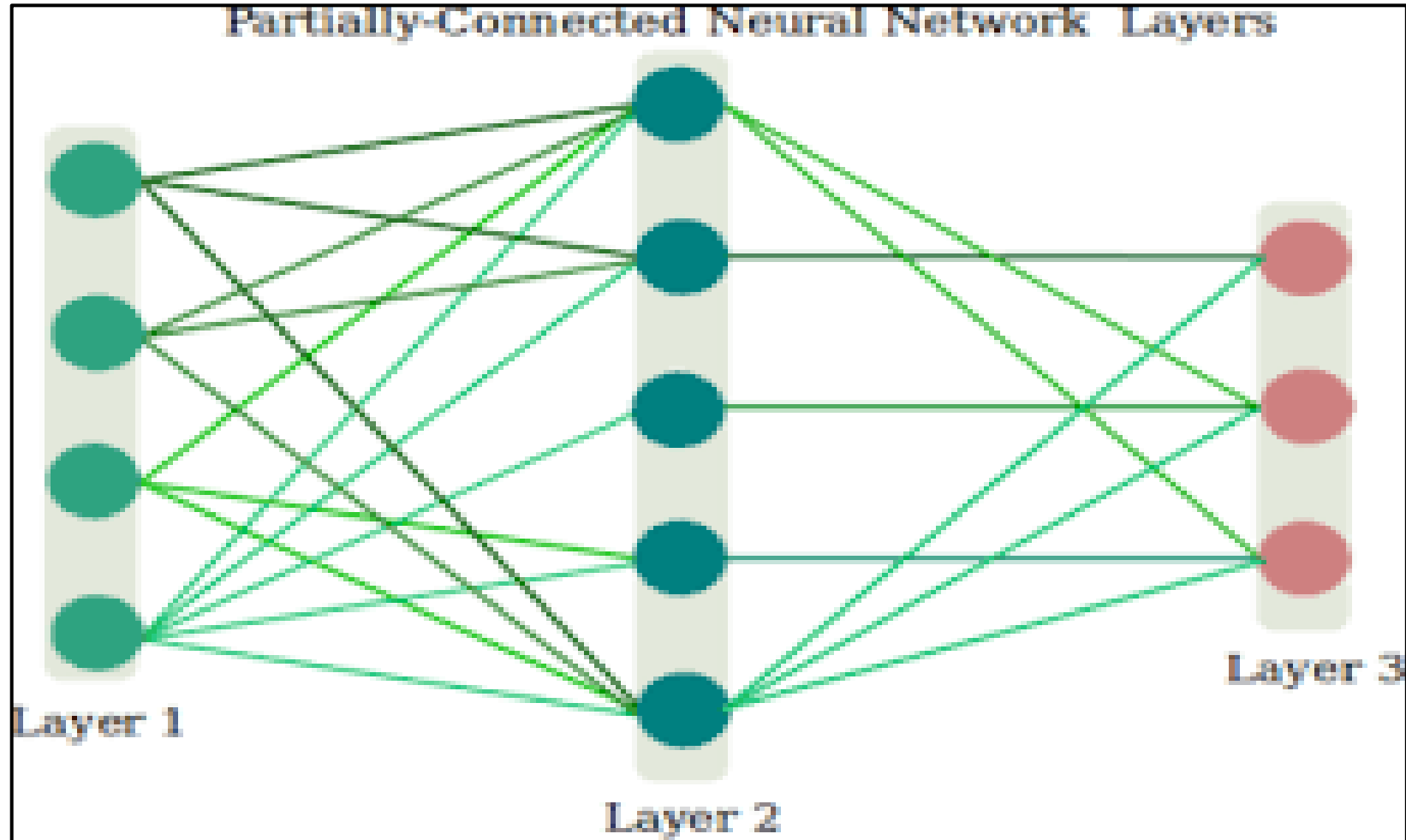
Partially Connected Layer

» In a Partially Connected Layer, each Unit **does not** receive input from **every Unit** of the previous layer

Example – Fully Connected ANN



Example – Partially Connected ANN



Feed Forward Network

Question

» Why ANN is called a Feed Forward Network?

Answer

» A Simple ANN is also called Feed Forward Network because **numbers** in the ANN, move in a **forward** direction

□ i.e. Input Layer == > Hidden Layer(s) == > Output Layer

» Note that **calculations** are performed at **each** Hidden and Output Unit

Weights – ANN



The **edges** between Input-Hidden Units, Hidden-Hidden Unit and Hidden-Output Unit contain the
» **Weights**



Recall

» Hypothesis (h) Representation in ANNs
□ **Combination of Weights between Units**



To Train ANN

» Initially, Weights are **randomly** assigned
» Normally, **small** Weights are assigned in a range of [-0.5 to +0.5]

ANNs have Black Box Representation



Model / h returned by Learner

» **Best** Combination of Weights between Units



ANNs are said to have Black Box Representation because

» **Useful knowledge** about learned concept (or Model / h) is **difficult** to **extract**

Importance – ANN Architecture



The **Architecture of ANN** plays an **important** role on the

- **Performance** and **computational cost** of ANN



Important **Parameters** to consider in **designing** ANN Architecture



Main Parameters

- No. of Input Units
- No. of Hidden Layers
 - » No. of Hidden Units at each Hidden Layer
- No. of Output Units
- Mathematical Function at each Hidden and Output Unit
- Weights between Units
- ANN will be Fully Connected or Not?

Types of Artificial Neural Networks

- ANN can be categorized based on number of hidden layers contained in ANN architecture

01

Two Layer Neural Network (Perceptron)

- Contains 0 hidden layers

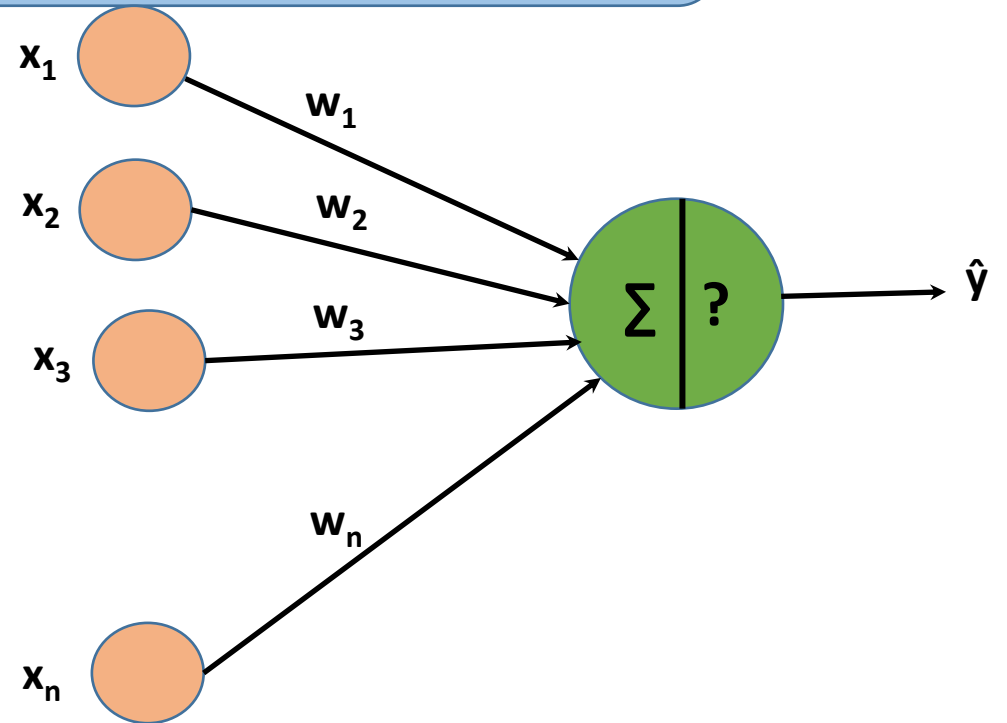
02

Multi Layer Neural Network

- Regular Neural Network
 - Contains 1 hidden layer
- Deep Neural Network
 - Contains >1 hidden layers

Two layer Artificial Neural Network (Perceptron)

- Multiple input nodes
- Single output node
 - Takes weighted sum of the inputs
 - Unit function calculates the output for the network



Two layer Artificial Neural Network (Perceptron)

Linear Function

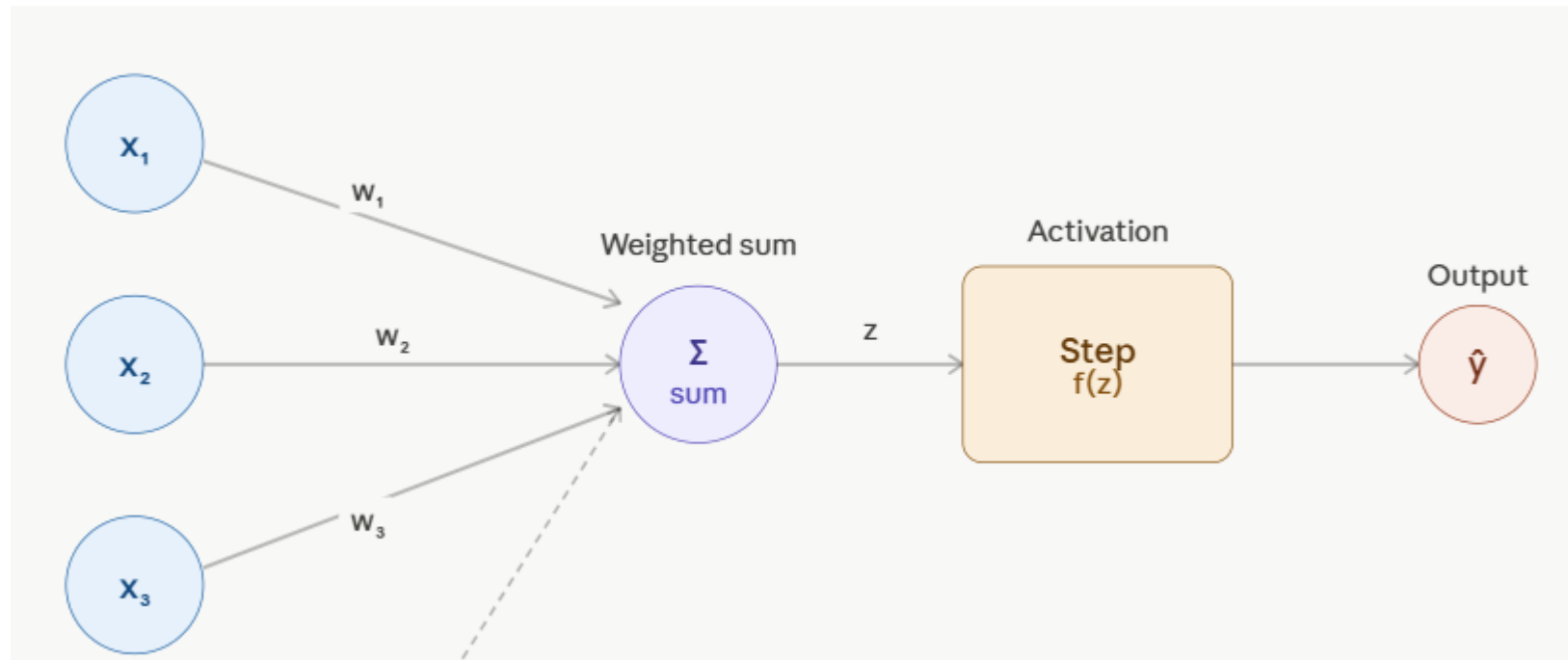
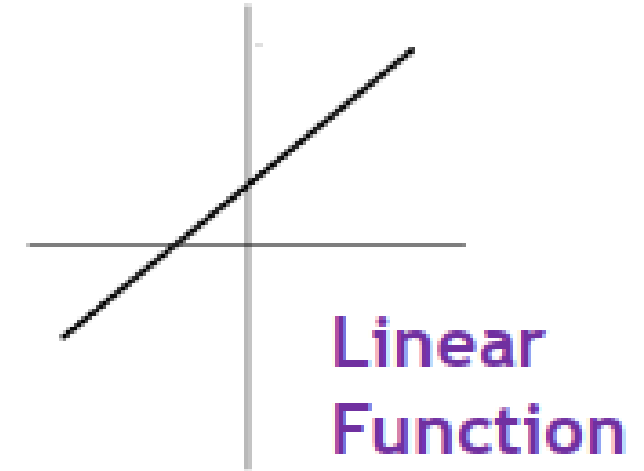
Weighted sum followed by an activation function

Activation Function Used:

The classic perceptron uses the **Step Function** (also called Heaviside function):

$$f(z) = 1 \text{ if } z \geq 0$$

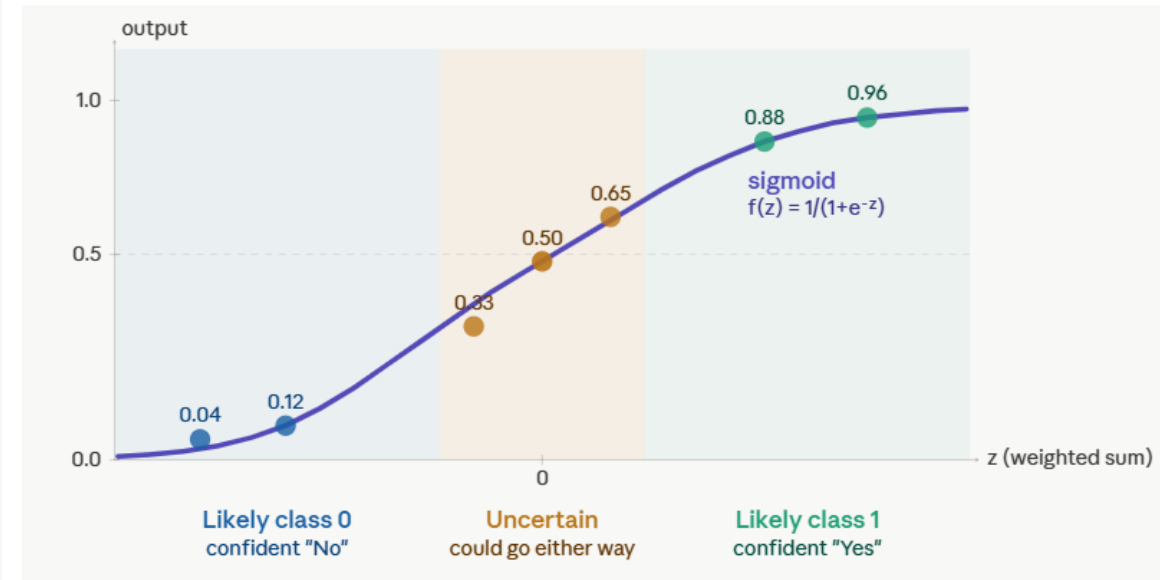
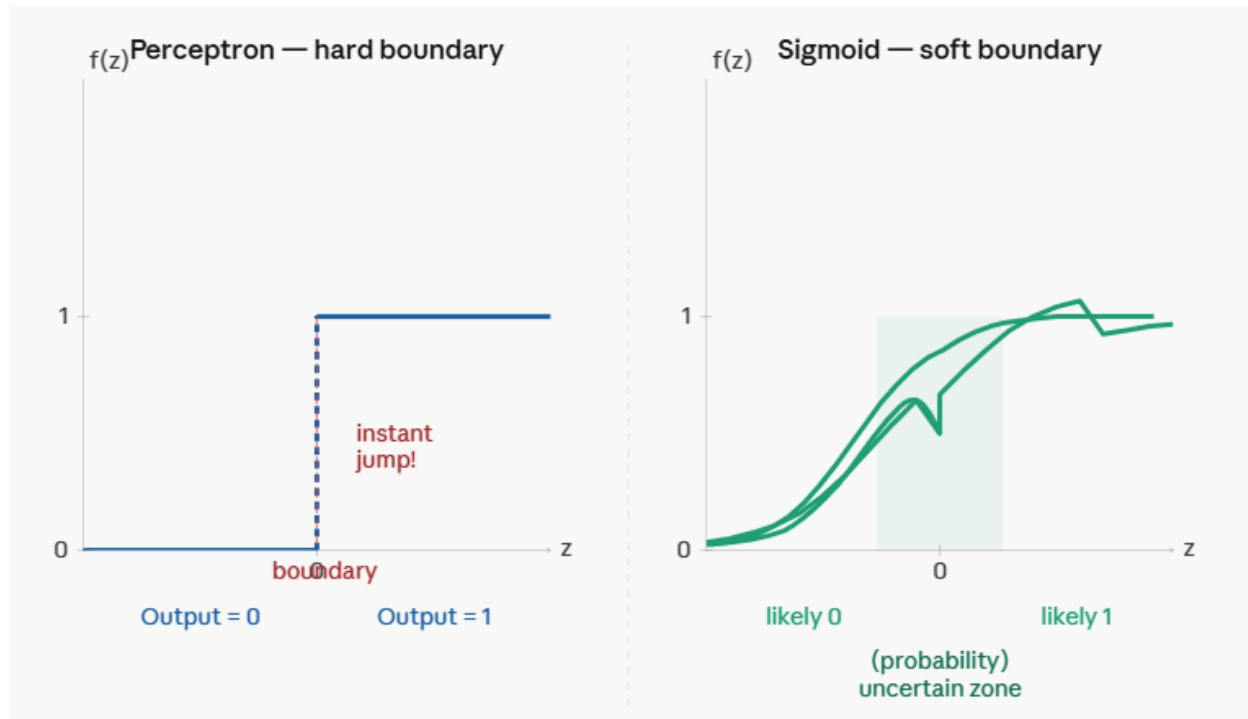
$$f(z) = 0 \text{ if } z < 0$$



Two layer Artificial Neural Network (Perceptron)

Perceptron model, which uses a hard decision boundary

A decision boundary is the line (or surface) that separates two classes — it's where the perceptron says "everything on this side = 0, everything on that side = 1."



Perceptron Example – Sample Data

Consider the Sample Data

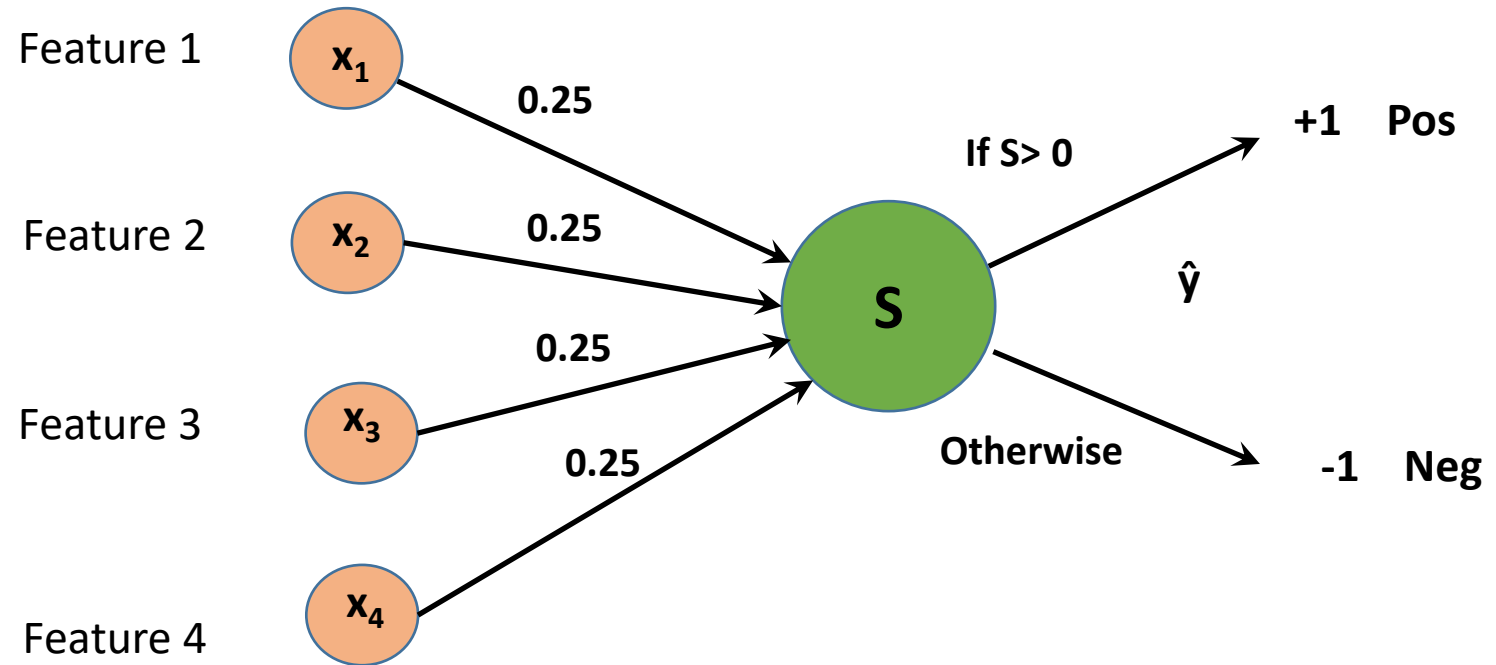
Instance No.	Input	Output
E_1	Bad	Negative
E_2	Good Product	Positive
E_3	Best product	Positive

Sample Data (Cont.)

Consider the Sample Data Features

Instance No.	Input				Output
	Bad	Product	Good	Nice	
E_1	1	0	0	0	-1
E_2		1	1	0	+1
E_3	0	1	0	1	+1

Perceptron Example



$$S = 0.25 * x_1 + 0.25 * x_2 + 0.25 * x_3 + 0.25 * x_4$$

Perceptron Example

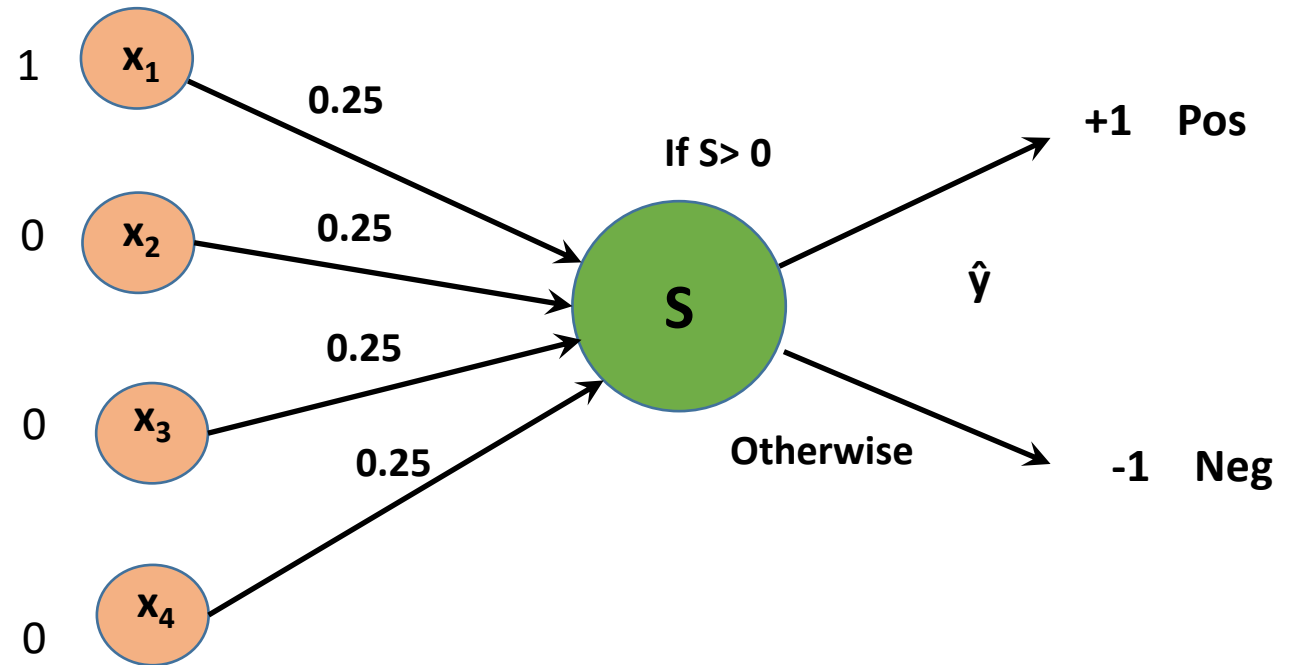
- Calculation (Step-1): (Instance No. 1)

- $x_1 = 1$
- $x_2 = 0$
- $x_3 = 0$
- $x_4 = 0$

$$S = 0.25 * (1) + 0.25 * (0) + 0.25 * (0) + 0.25 * (0) = 0.25$$

- $0.25 > 0$, so the output of ANN is +1

- So the image is categorized as "Pos"
- Target : "Neg"



Learning Algorithm

- **Weights are randomly initialized.**
- **For each training example E**
 - Calculate the observed output from Perceptron, $o(E)$
 - If the target output $t(E)$ is different to $o(E)$
 - Then update all the weights so that $o(E)$ becomes closer to $t(E)$
- **This process is done for every example**
- **It is not necessary to stop when all examples are used.**
 - Repeat the cycle again (an epoch) until network produces the correct output

Perceptron Training Rule (How to update weights)

- When $t(E)$ is different from $o(E)$
 - Add Δ_i to weight w_i
 - Where change $\Delta_i = \eta(t(E) - o(E)) x_i \rightarrow \eta$ is learning rate (Usually very small value)
 - Do this for every weight in the network
 - Let $\eta=0.1$

Calculating the error values

$$\begin{aligned}\Delta_1 &= \eta (t(E) - o(E)) * x_1 \\ &= 0.1 (-1 - (+1)) * 1 = -0.2\end{aligned}$$

$$\begin{aligned}\Delta_2 &= \eta (t(E) - o(E)) * x_2 \\ &= 0.1 (-1 - 1) * 0 = 0\end{aligned}$$

$$\begin{aligned}\Delta_3 &= \eta (t(E) - o(E)) * x_3 \\ &= 0.1 (-1 - 1) * 0 = 0\end{aligned}$$

$$\begin{aligned}\Delta_4 &= \eta (t(E) - o(E)) * x_4 \\ &= 0.1 (-1 - 1) * 0 = 0\end{aligned}$$

Calculating the New Weights

$$w'_1 = w_1 + \Delta_1 = 0.25 + (-0.2) = 0.05$$

$$w'_2 = w_2 + \Delta_2 = 0.25 + 0 = 0.25$$

$$w'_3 = w_3 + \Delta_3 = 0.25 + 0 = 0.25$$

$$w'_4 = w_4 + \Delta_4 = 0.25 + 0 = 0.25$$

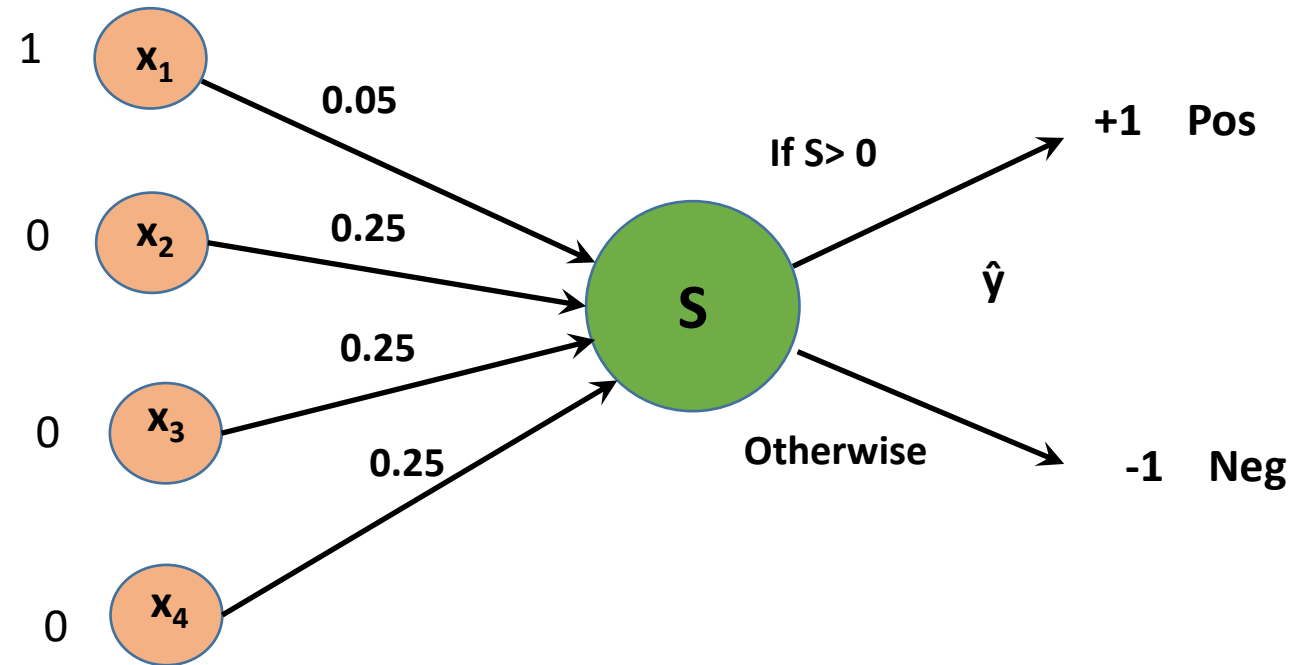
Perceptron Example

- Calculation (Step-2):

- $x_1 = 1$
- $x_2 = 0$
- $x_3 = 0$
- $x_4 = 0$

$$S = 0.05 * (1) + 0.25 * (0) + 0.25 * (0) + 0.25 * (0) = 0.05$$

- $0.05 > 0$, so the output of ANN is +1
- So the image is categorized as "Pos"
- Target : "Neg"



Perceptron Training Rule (How to update weights)

- When $t(E)$ is different from $o(E)$
 - Add Δ_i to weight w_i
 - Where $\Delta_i = \eta(t(E) - o(E)) x_i \rightarrow \eta$ is learning rate (Usually very small value)
 - Do this for every weight in the network
 - Let $\eta=0.1$

Calculating the error values

$$\begin{aligned}\Delta_1 &= \eta (t(E) - o(E)) * x_1 \\ &= 0.1 (-1 - 1) * 1 = -0.2\end{aligned}$$

$$\begin{aligned}\Delta_2 &= \eta (t(E) - o(E)) * x_2 \\ &= 0.1 (-1 - 1) * 0 = 0\end{aligned}$$

$$\begin{aligned}\Delta_3 &= \eta (t(E) - o(E)) * x_3 \\ &= 0.1 (-1 - 1) * 0 = 0\end{aligned}$$

$$\begin{aligned}\Delta_4 &= \eta (t(E) - o(E)) * x_4 \\ &= 0.1 (-1 - 1) * 0 = 0\end{aligned}$$

Calculating the New Weights

$$w'_1 = w_1 + \Delta_1 = 0.05 - 0.2 = -0.15$$

$$w'_2 = w_2 + \Delta_2 = 0.25 + 0 = 0.25$$

$$w'_3 = w_3 + \Delta_3 = 0.25 + 0 = 0.25$$

$$w'_4 = w_4 + \Delta_4 = 0.25 + 0 = 0.25$$

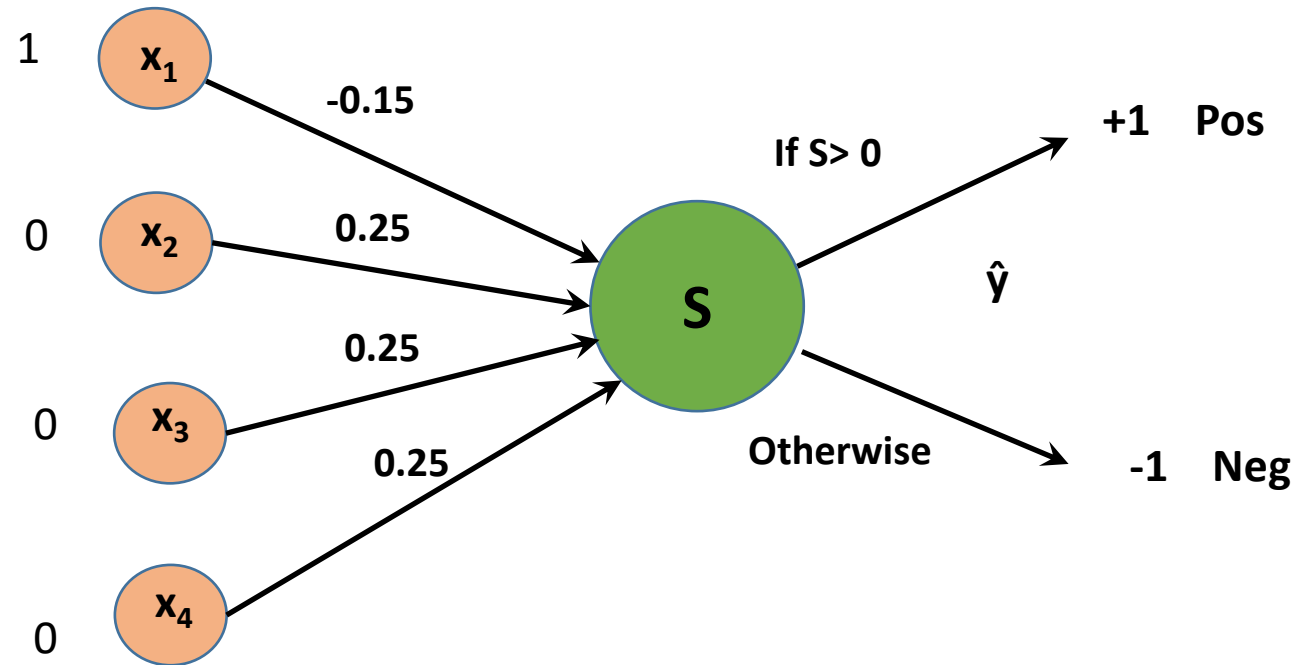
Perceptron Example

- Calculation (Step-3):

- $x_1 = 1$
- $x_2 = 0$
- $x_3 = 0$
- $x_4 = 0$

$$S = -0.15 * (1) + 0.25 * (0) + 0.25 * (0) + 0.25 * (0) = -0.15$$

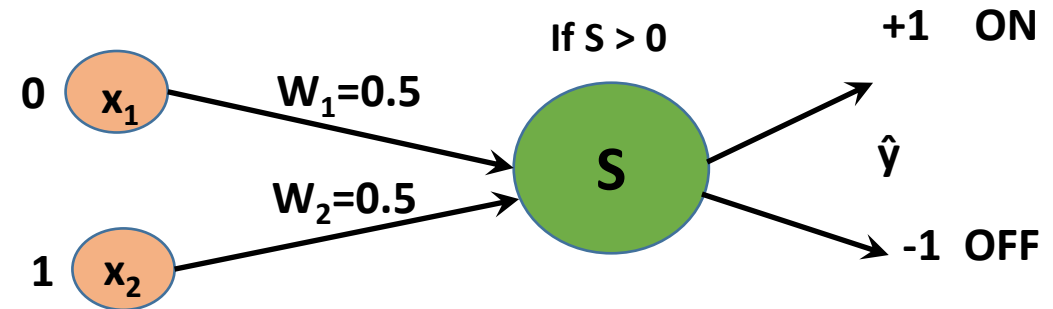
- $-0.15 < 0$, so the output of ANN is -1
- So the image is categorized as “Neg”
- Target : “Neg”



Another Example (AND)

x_1	x_2	$x_1 \text{ AND } x_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $x_1 = 0$
- $x_2 = 1$,
- $\eta = 0.1$
- $t(E) = -1$

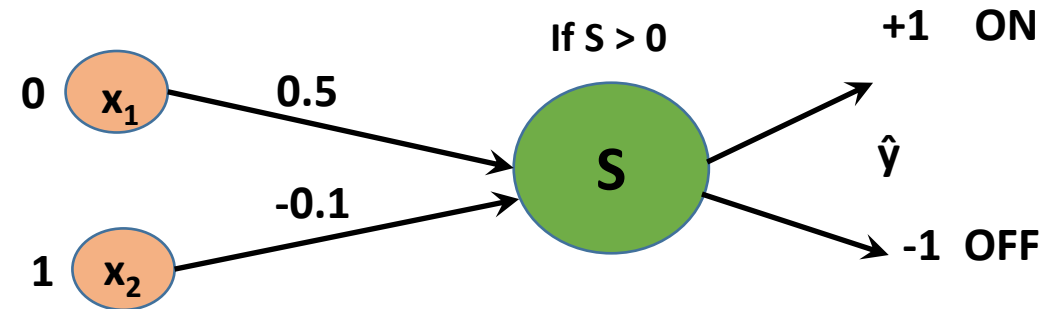


Weights	Step-1	Step-2	Step-3	Step-4
w_1	0.5	0.5	0.5	0.5
w_2	0.5	0.3	0.1	-0.1
Weighted Sum	0.5	0.3	0.1	-0.1
Observed Output	+1	+1	+1	-1

Another Example (AND)

X_1	X_2	$X_1 \text{ AND } X_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $X_1 = 1$
- $X_2 = 0$,
- $\eta = 0.1$
- $t(E) = -1$

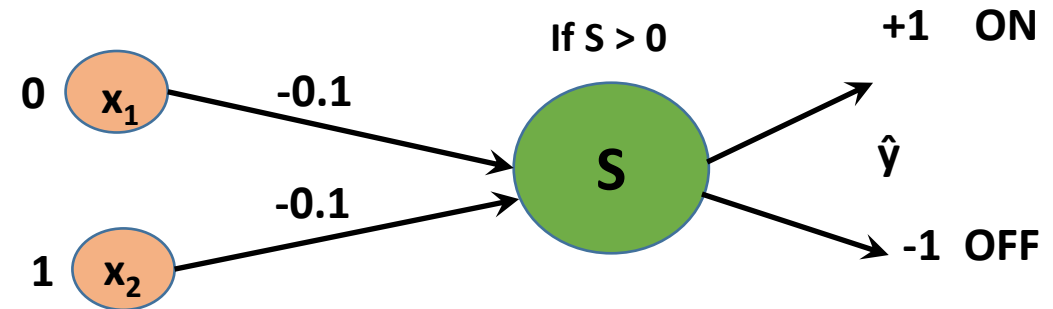


Weights	Step-1	Step-2	Step-3	Step-4
w1	0.5	0.3	0.1	-0.1
w2	-0.1	-0.1	-0.1	-0.1
Weighted Sum	0.5	0.3	0.1	-0.1
Observed Output	+1	+1	+1	-1

Another Example (AND)

X_1	X_2	$X_1 \text{ AND } X_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $X_1 = 1$
- $X_2 = 1$,
- $\eta = 0.1$
- $t(E) = +1$

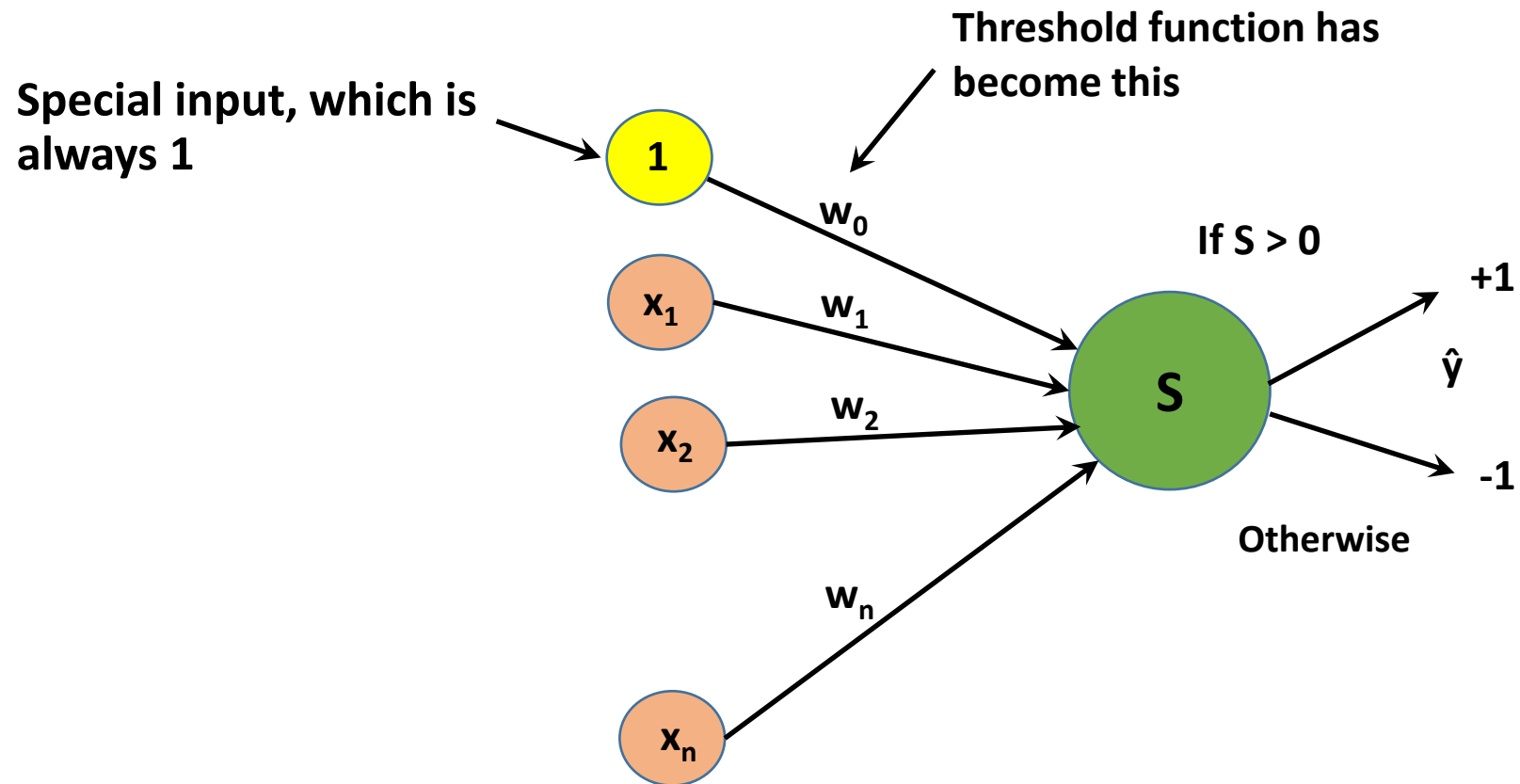


Weights	Step-1	Step-2
w1	-0.1	0.1
w2	-0.1	0.1
Weighted Sum	-0.2	0.2
Observed Output	-1	+1

Why Bias is Required....?

- **Improves model performance and learning**
- **Bias helps the model better fit the training data.**
- **It often helps the model converge faster during training.**
- **Technically, it lets the model learn more complex patterns.**

New Representation for Perceptron



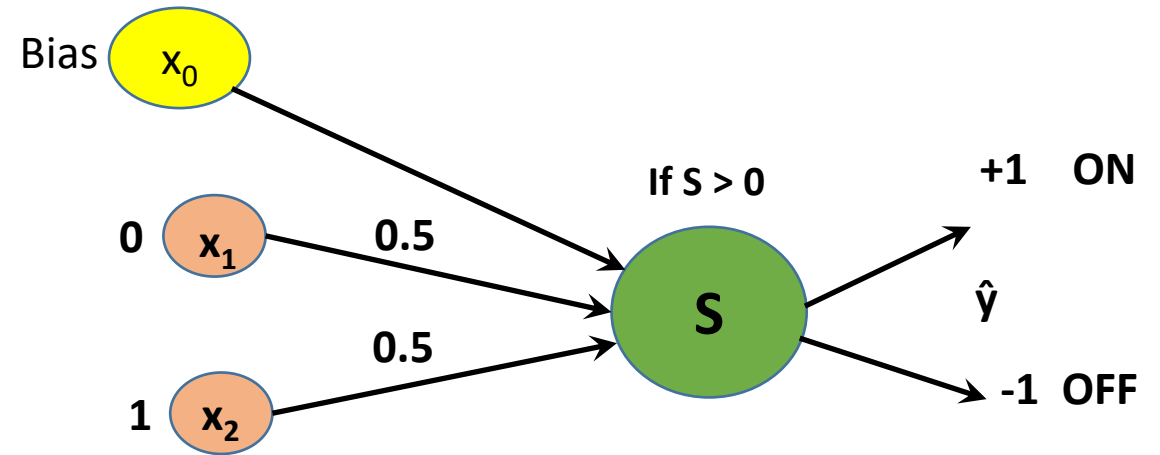
$$S = w_0 + w_1 * x_1 + w_2 * x_2 \dots \dots \dots w_n * x_n$$

Use of Bias

- Bias is just like an intercept added in a linear equation.

$$\text{output} = \text{sum}(\text{weights} * \text{inputs}) + \text{bias}$$

- The output is calculated by multiplying the inputs with their weights and then passing it through an activation function like the Sigmoid function, etc. Here, bias acts like a constant which helps the model to fit the given data.

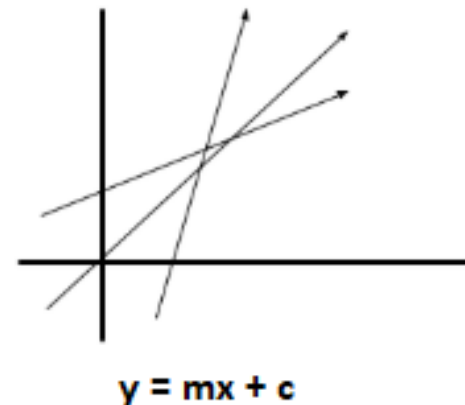
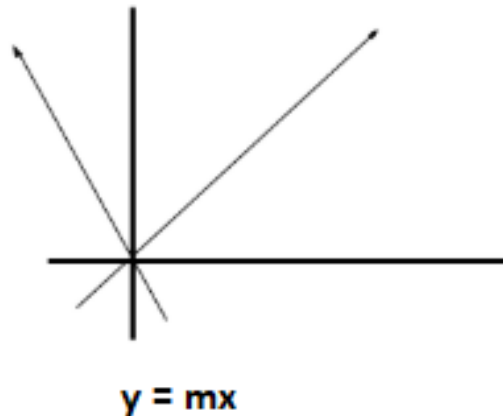


Use of Bias

- A simpler way to understand bias is through a constant c of a linear function

$$y = mx + c$$

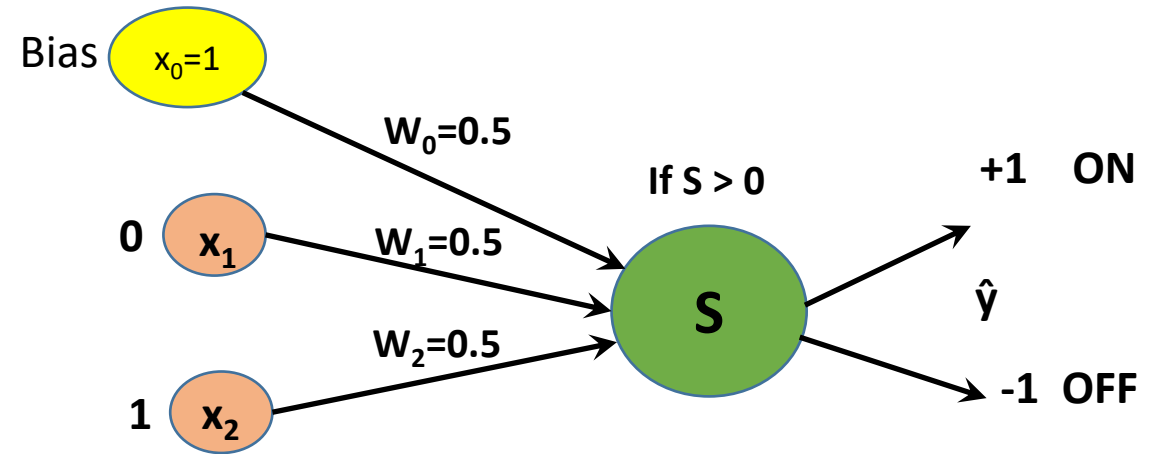
- It allows us to move the line down and up fitting the prediction with the data better. If the constant c is absent then the line will pass through the origin $(0, 0)$ and we will get a poorer fit.



Example (AND) with Bias

X_1	X_2	$X_1 \text{ AND } X_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $X_1 = 0$
- $X_2 = 1$,
- $\eta = 0.1$
- $t(E) = -1$

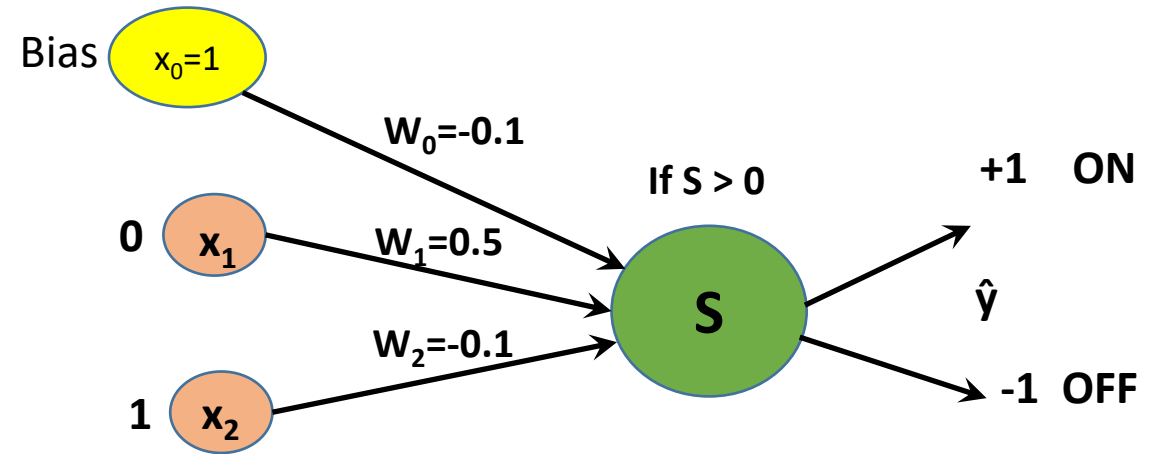


Weights	Step-1	Step-2	Step-3	Step-4
w_0	0.5	0.3	0.1	-0.1
w_1	0.5	0.5	0.5	0.5
w_2	0.5	0.3	0.1	-0.1
Weighted Sum	1	0.6	0.2	-0.2
Observed Output	+1	+1	+1	-1

Example (AND) with Bias

X_1	X_2	$X_1 \text{ AND } X_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $X_1 = 1$
- $X_2 = 0$,
- $\eta = 0.1$
- $t(E) = -1$

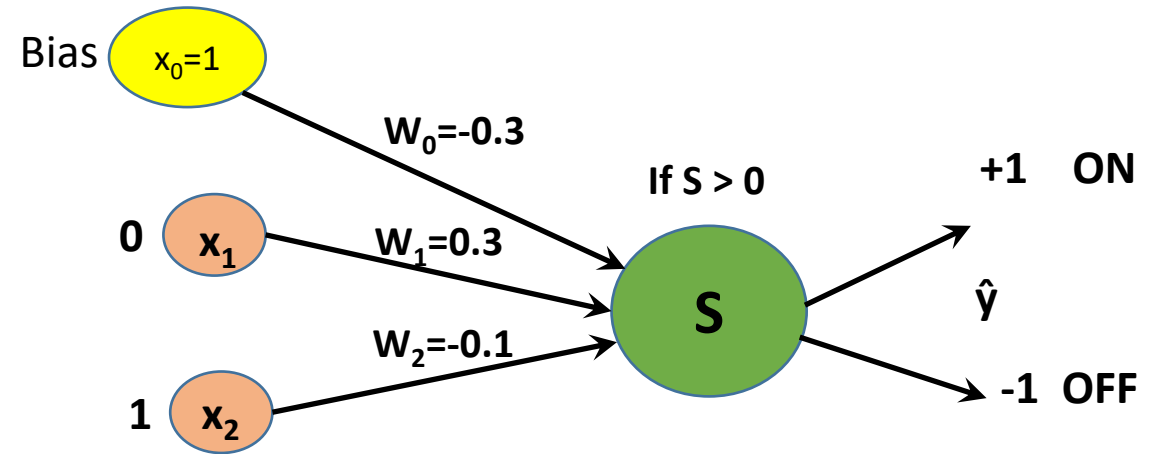


Weights	Step-1	Step-2
w_0	-0.1	-0.3
w_1	0.5	0.3
w_2	-0.1	-0.1
Weighted Sum	0.4	0
Observed Output	+1	-1

Example (AND) with Bias

X_1	X_2	$X_1 \text{ AND } X_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $X_1 = 1$
- $X_2 = 1$,
- $\eta = 0.1$
- $t(E) = +1$



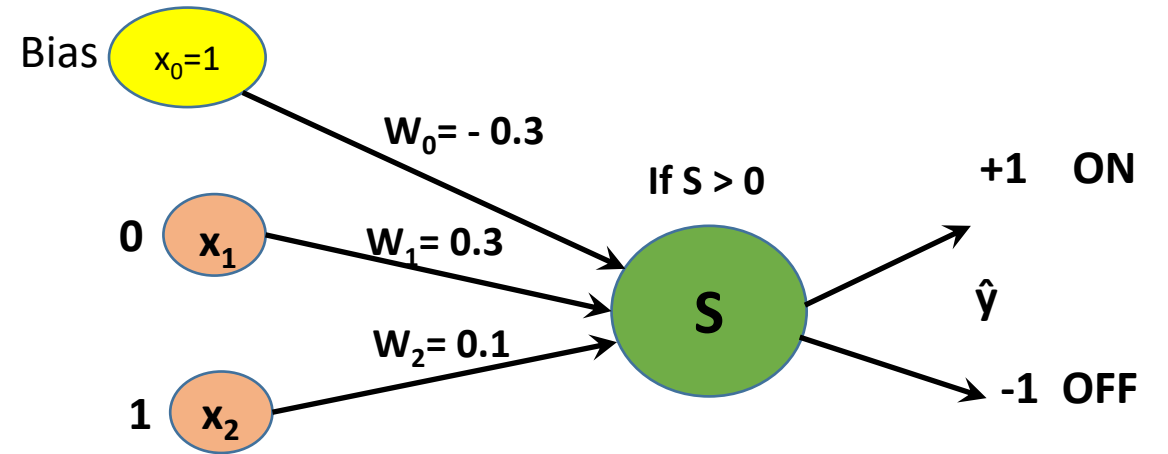
Weights	Step-1	Step-2
w_0	-0.3	-0.1
w_1	0.3	0.5
w_2	-0.1	0.1
Weighted Sum	0	0.5
Observed Output	-1	+1

Example (AND) with Bias

After 2 Epochs

X_1	X_2	$X_1 \text{ AND } X_2$
0	0	0
0	1	0
1	0	0
1	1	1

- $X_1 = 0$
- $X_2 = 1$,
- $\eta = 0.1$
- $t(E) = -1$



Final Weights

Weights	
w_0	- 0.3
w_1	0.3
w_2	0.1

Learning in Perceptron



Need To Learn

- Both the **weights** between input and output units
- And the value for the **threshold**
- **Make Calculations easier by:**
 - Thinking of the threshold as a weight from a special input unit where the output from the unit is always 1
- **Exactly the same result:**
 - But we only have to worry about learning weights

Limitations of Perceptron

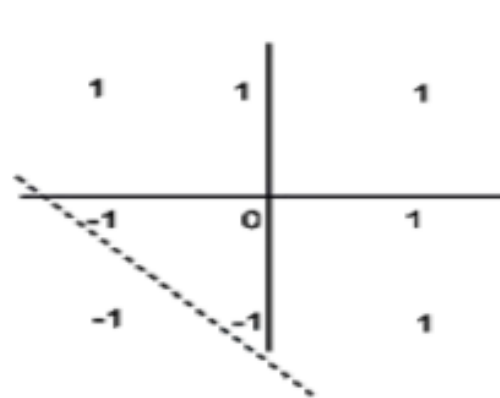
- Cannot learn some simple Boolean functions
- Can only learn linearly separable Boolean functions

Linearly Separable

- Can use a line (dotted) to separate +1 and -1



AND VALUES



OR VALUES



XOR VALUES

THANKS